

单 Panthera 机械臂圆柱入槽： 从手势遥操作到仿真 VLA 实验

Panthera 项目阶段性研究报告（修订版）· 2026 年 9 月 15 日

摘要：本文记录一项仍在进行中的机器人实验：让一台 Panthera 六轴机械臂用夹爪抓起竖直圆柱，再把圆柱插入凹槽。研究分为两个阶段。第一阶段用单目摄像头识别人手臂动作，并通过 ROS 2 控制真实机械臂；第二阶段在物理仿真中训练视觉—语言—动作模型（Vision-Language-Action model, VLA）。前一阶段提供机器人通信、连续控制和安全停止等基础设施；当前 VLA 的训练数据全部来自仿真程序生成的专家示范。仿真阶段共收集 128 条成功示范，并用 OpenVLA-OFT 进行监督微调。实验发现，较低的离线动作误差尚未带来连续任务成功。把预测长度从 5 帧增加到 25 帧后，模型能够抓取和搬运部分圆柱，但插入和松爪仍容易失败。为分析这一问题，我们还评测了“VLA 负责抓取和搬运、固定程序负责最后插入”的组合系统。在 16 个反复用于调试的开发场景中，组合系统有 11 条轨迹曾至少一次满足预设成功条件，即 68.75%。这属于开发集上的阶段结果；轨迹结束时的稳定成功数量尚未报告。最终仿真测试和真实机械臂上的 VLA 实验均未开始。

关键词：具身智能；视觉—语言—动作模型；Panthera；OpenVLA-OFT；RoboTwin；模仿学习；仿真到现实

1 引言

传统机械臂程序通常由工程师直接规定关节角、目标位置和执行顺序。具身智能研究希望机器人能根据相机图像、文字指令和自身关节状态选择动作。RT-2 把同时处理视觉、语言和机器人动作的模型称为视觉—语言—动作模型（VLA）[1]。OpenVLA 是一个公开的 70 亿参数 VLA；其正式论文报告使用约 97 万条机器人示范进行预训练[2]。Open X-Embodiment 等工作则尝试把不同机器人的数据整理成可以共同训练的形式[14]。

“圆柱入槽”包含一连串相互影响的步骤：机器人先观察物体，再移动夹爪、夹稳圆柱、保持姿态完成搬运，最后处理圆柱与槽壁的接触。任一步骤的误差都会改变下一张图像以及后续动作。因此，本文同时检查模型预测、机械臂运动、物理接触和任务结果；训练损失只是其中一项诊断数据。

项目早期做过基于人体关键点的单目遥操作，后来才开始当前 VLA 实验。两者共享同一台机械臂和一部分控制软件，但输入和目的不同：遥操作把人的动作映射到机器人；当前 VLA 根据任务画面自行输出动作。本文介绍早期工作，用于说明机器人接口和安全控制的来源。MediaPipe 仅用于早期遥操作。

本文回答三个问题：第一，怎样在仿真中建立一台 Panthera 完成圆柱入槽的任务并生成训练数据；第二，动作预测在什么情况下会从离线误差较低变成连续执行失败；第三，在最终测试和真实机械臂实验之前，应该保存哪些证据并设置哪些停止条件。

2 相关工作及其方法

本节说明后文引用的方法具体做了什么。表 1 先给出它们之间的区别，随后列出本项目实际采用的部分。每项参考工作都只在表中注明的支持本文设计，项目仍会根据实验结果调整或弃用这些方法。

表 1 相关工作的方法和它们与本项目的关系

工作	核心方法	本项目采用的部分
RT-2 [1]	把机器人动作写成离散符号，使视觉语言模型可以像生成文字一样生成动作；训练时同时使用互联网视觉语言任务和机器人轨迹。	只借用“图像、语言和动作可以由同一策略建模”的 VLA 问题定义。
Open X-Embodiment [14]	汇集不同机构、不同机器人和不同任务的数据，并研究用这些异构数据共同训练机器人策略。	解释 OpenVLA 预训练数据的来源；本项目自己的 Panthera 数据仍单独定义动作含义。
OpenVLA [2]	把两个视觉编码器与 70 亿参数语言模型结合，并把连续动作量化成离散区间后作为动作符号输出。	使用其公开预训练模型作为初始模型，但不沿用离散动作输出。
OpenVLA-OFT [11]	增加机器人自身状态输入、连续动作输出头、动作分块和并行解码，使一次推理能够产生多个连续动作。	使用连续 L1 动作头，输出 25 个七维 Panthera 动作；不使用其扩散动作头。
ACT [13]	用条件变分自编码器和 Transformer 一次预测一段动作，并可融合相邻时刻预测的重叠动作。	作为动作分块的相关例子；当前实现采用 OpenVLA-OFT，未使用 ACT 的网络和时间融合。
Diffusion Policy [15]	从随机噪声开始，经过多次去噪生成一段与当前观察相符的连续动作。	尚未使用；以后可以作为不同于 L1 直接回归的动作生成基线。
DAgger [12]	先让正在学习的策略运行，再请专家为策略实际到达的状态标注正确动作，把新样本加入训练集。	尚未使用；它是以后补充失败状态训练数据的候选方法。
基于图像的视觉伺服 [16]	计算当前图像特征与目标图像特征之间的误差，再用图像雅可比矩阵把误差转换成机器人速度。	尚未使用；以后可作为不依赖大型 VLA 的传统控制基线。
域随机化 [9]	训练时随机改变物体、光照、相机或物理参数，使模型不依赖单一仿真画面。	对光照、背景、物体位置和小幅相机偏差做限定范围内的随机变化。
RoboTwin 与 SAPIEN [6,7]	RoboTwin 组织任务、示范生成和评测环境；SAPIEN 计算刚体、关节、碰撞与接触。	在其中加入单 Panthera 资产、圆柱入槽任务和手机视角相机，并实际执行专家轨迹和模型动作。

RLinf [8]	把模型推理、仿真环境和计算设备组织成可并行运行的工作单元。其论文重点是大规模强化学习系统，但代码也提供 VLA 在 RoboTwin 中的并行评测入口。	只用于加载训练完成的模型并运行多环境闭环评测；本文的监督微调不由 RLinf 执行。
RLDS [10]	按“整条轨迹—轨迹中的连续采样帧”组织机器人数据。	用来保存图像、七维状态和七维动作；RLDS 本身不训练模型。

2.1 从视觉语言模型到机器人动作

RT-2 把动作数字转换成语言模型词表中的离散符号。模型接收图像和任务文字后，自回归地生成这些动作符号。训练时混合互联网视觉问答数据和机器人轨迹，使已有的视觉语言知识能够影响机器人决策[1]。

Open X-Embodiment 收集了来自多种机器人的数据。不同数据集的相机、关节和动作定义并不天然相同，因此这项工作还要为各数据源保留对应的动作空间和任务说明，并研究跨机器人联合训练[14]。OpenVLA 使用其中约 97 万条机器人示范进行预训练。它用 DINOv2 和 SigLIP 两种视觉特征描述图像，再把视觉特征与语言指令送入 70 亿参数语言模型。原始 OpenVLA 将每个连续动作维度量化到离散区间，最后生成动作符号[2]。

原始 OpenVLA 中常见的七维动作与本文的七维动作只是数量相同，含义不一定相同。前者通常表示末端位置变化、末端旋转变化和夹爪；本文重新配置的输出表示六个 Panthera 关节的绝对目标角度和一个夹爪开度。数据格式和动作头配置共同决定输出维度，本项目把它明确设为七维。

2.2 OpenVLA-OFT 怎样修改 OpenVLA

原始 OpenVLA 逐个生成离散动作符号，这种方式会增加机器人控制的推理时间，也会丢失动作量化区间内部的数值差异。OpenVLA-OFT 在预训练模型后增加专门的动作输出模块：它可以直接回归浮点动作，也可以使用扩散模型生成动作分布；同时让模型一次并行预测多个未来动作，并可把机器人当前状态作为额外输入[11]。

本项目只采用其中的连续 L1 回归方案。模型读取一张图像、任务文字和当前七维状态，动作头一次输出 25×7 个浮点数。训练目标是减小这些预测值与专家动作之间的平均绝对误差。关节限位、碰撞距离等信息原则上可以继续作为模型输入，但本文没有这样配置；模型只读取 RGB 图像和七维机器人状态。即使以后加入这些信息，学习模型给出的仍是预测值，不能保证它永远遵守约束。因此本文把关节限位、碰撞检查和停止逻辑保留在确定性执行层，把抓取与插入是否成功交给仿真器和评测程序判断。

2.3 动作分块、数据分布偏移与其他策略

ACT 是另一种动作分块模型。它使用条件变分自编码器表示示范中不同的动作风格，再由 Transformer 输出动作块；运行时还可以对相邻预测中指向同一时刻的动作加权平均，从而减小抖动[13]。本文借它说明动作分块已有先例，当前实现仍采用 OpenVLA-OFT，也未采用 ACT 的时间融合。

Diffusion Policy 从随机动作序列开始，根据当前观察反复去除噪声，最后得到一段连续动作[15]。这种方法可以表达同一场景下的多种可行动作，但需要多次去噪计算。本文选择了计算路径更直接的 L1 回归动作头，扩散方案留作以后比较。

行为克隆只学习专家访问过的状态。当模型自己运行时，小错误会把机器人带到专家数据没有覆盖的位置，后续错误因而继续扩大。Dagger 的解决方法是让当前策略运行，再由专家为这些新状态提供正确动作，并反复把新数据加入训练集[12]。本文目前只使用最初收集的成功示范，没有执行 Dagger，所以它只能作为下一阶段候选。

传统的基于图像的视觉伺服不需要理解开放式语言。工程师先定义圆柱、槽口等图像特征和目标位置，控制器计算当前特征误差，再用图像雅可比矩阵求出减小误差所需的机器人速度[16]。它依赖可靠的目标检测和相机几何，但能为固定任务提供清楚、可解释的对照。本文尚未实现这一基线。

2.4 仿真与数据方法

SAPIEN 是物理仿真器，负责计算关节运动、重力、碰撞和接触力[7]。RoboTwin 在物理仿真器之上定义机器人任务、专家数据生成和评测流程，并使用结构化域随机化改变场景[6]。本项目借用这一框架，但重新加入单台 Panthera 和七维动作接口；RoboTwin 原有的双臂任务定义不属于本文实验。

域随机化通过在训练中主动改变视觉或物理条件，降低模型只记住一种仿真场景的风险[9]。它只能覆盖设置过的变化范围，不能修复错误的圆柱尺寸、槽宽或相机位置。RLDS 则只是数据格式：它保存每条轨迹的连续图像、状态、动作和任务文字[10]，并不进行仿真、训练或控制。

3 任务与基础概念

当前任务只使用一台 Panthera 六轴机械臂。机械臂末端安装一个开合夹爪；桌面上有一个竖直黄色圆柱和一个带孔的黄色槽座。目标是夹起圆柱，把它移到孔的上方，沿竖直方向插入，然后松开夹爪。早期代码曾支持双臂机器人，但本文的训练、评测和结论均不使用双臂数据。

在每个采样时刻，程序用七个数表示机器人状态：

$$s_t = [q_1, q_2, q_3, q_4, q_5, q_6, g]_t$$

其中 $q_1 \dots q_6$ 是六个旋转关节的角度， $g \in [0,1]$ 表示夹爪开度。模型也输出七个数，含义是六个关节下一步应到达的绝对角度和夹爪目标开度。正运动学根据关节角计算夹爪的位置和朝向；逆运动学从期望的夹爪位置和朝向反求关节角；轨迹规划在起点和终点之间生成机器人能够连续执行的中间状态[3]。

表 2 阅读后文需要的基本术语

术语	本文中的定义
采样帧 (step)	某一时刻保存的一组相机图像、机器人状态和目标动作。
一条轨迹 (episode)	从场景复位开始，到成功、失败或超时结束的一整段连续采样帧。
本体状态 (proprioception)	机器人对自身状态的测量。本文只使用六个关节角和夹爪开度，不包含物体位置或碰撞距离。
RGB 图像	普通彩色图像，由红、绿、蓝三个颜色通道组成。本文的策略不使用深度图或点云。

程序化专家 (oracle)	能读取仿真中的精确物体位置并按预先编写的步骤完成任务的控制程序。它负责验证任务和生成示范；学习模型随后模仿这些示范。
模仿学习	用专家示范训练模型，使模型在看到相似图像和机器人状态时预测专家动作。
监督微调 (SFT)	在预训练模型基础上，用“观察—专家动作”配对继续训练。本文采用这种模仿学习方法；通过奖励试错的强化学习属于另一条路线。
随机种子 (seed)	控制物体位置、光照等随机变量的整数。同一程序使用相同 seed 时应生成相同场景。
动作块	模型一次预测的连续多个未来动作。本文的“25×7”表示未来 25 帧、每帧 7 个动作数值。
动作归一化	根据训练数据中每个动作维度的数值范围，把不同量级的关节和夹爪值变换到便于训练的范围。执行前必须用同一组统计量还原。
L1 误差	逐项计算预测动作与专家动作差值的绝对值，再求平均。数值越小表示在给定数据上的动作越接近专家。
闭环执行	机器人执行一小段预测动作后重新拍摄图像，再根据新图像预测下一段动作。
训练检查点 (checkpoint)	训练到某一步时保存的模型参数。“15k”表示训练或续训过程记录到 15,000 个优化步骤。
GPU 与 worker	GPU 是执行模型计算的图形处理器；worker 是 RLinf 启动的独立工作进程，例如负责模型推理的进程或负责仿真环境的进程。
LoRA	一种节省显存的微调方法。它冻结原有大模型权重，在部分线性层旁增加较小的可训练低秩矩阵。
MPLib	本文调用的运动规划库。它根据机器人模型、起点、目标和碰撞关系搜索一条关节路径。
Ray	RLinf 用来启动和管理多个本地工作进程的分布式运行组件。评测结束后应确认相关进程已经退出。
URDF 与 SRDF	URDF 描述机器人的连杆、关节和几何；SRDF 补充规划组和允许忽略的碰撞关系。学习模型通过仿真环境间接使用这些机器人描述。
开发集与最终测试集	开发集可以反复查看并据此修改方案；最终测试集在方案冻结后只运行一次，用来减少针对已知场景调参造成的偏差。
确定性末端技能	人工编写的最后一段动作程序。输入相同时，它按同样步骤执行定姿、对齐、插入和松爪，不通过学习产生这些动作。
仿真到真实 (sim-to-real)	把仿真中训练和验证的方法迁移到真实机器人。本文尚未完成这一步。

4 系统架构

图 1 给出当前仿真实验一次决策所经过的步骤。仿真相机提供 RGB 图像，任务文字固定为“将圆柱插入槽中”，仿真器同时提供六个关节角和夹爪开度。OpenVLA-OFT 根据这三类输入预测动作块。程序执行动作块的一部分后，仿真器计算新的图像和机器人状态，再开始下一轮预测。满足接管条件时，最后的插入动作改由确定性末端技能执行。

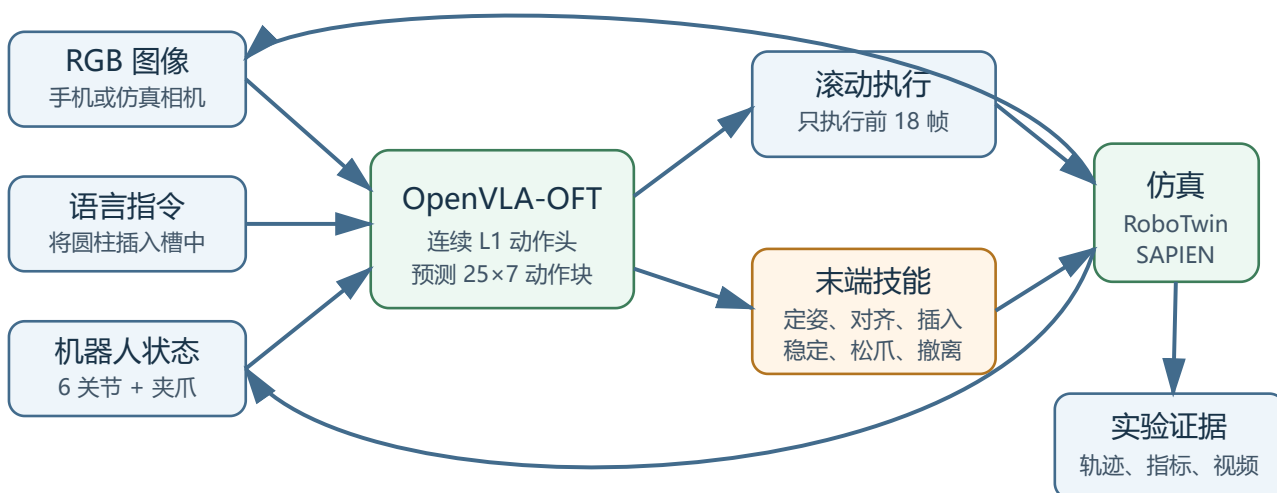


图 1 当前单 Panthera 仿真闭环架构。VLA 负责抓取和搬运，确定性末端技能负责最后的接触过程。

4.1 遥操作基础设施

在开始 VLA 训练之前，项目先用手臂动作控制过真实 Panthera。MediaPipe 从普通摄像头图像中检测肩、肘、腕和手部关键点[5]；ROS 2 负责在视觉程序、运动计算程序和机器人驱动程序之间传递数据[4]。这项早期工作验证了关节反馈、坐标换算、数据录制以及非阻塞控制。它还规定：视觉数据过期或跟踪丢失后停止接受新目标，数据恢复后也必须由操作者重新启用。当前 VLA 没有使用人体关键点，但以后连接真实机械臂时仍会复用这些机器人接口和停止规则。

4.2 仿真和训练基础设施

RoboTwin 提供任务、数据采集和评测框架[6]，其下层的 SAPIEN 负责计算机械臂、圆柱和槽座的运动、碰撞与接触[7]。RoboTwin 上游主要面向双臂任务，而且没有本文所需的 Panthera 资产，因此本项目从 Panthera 官方 ROS 2 模型生成仿真资产，再增加一个单臂、单夹爪的竖直圆柱入槽任务。所有对外状态和动作都使用前述七个数。

训练和评测使用不同程序。监督微调由 OpenVLA-OFT 的 finetune.py 执行，并由 PyTorch 的 torchrun 在三张 GPU 上启动[17]。训练完成后，RLinf 加载模型，在多张 GPU 上并行启动 RoboTwin 环境，反复完成“读取图像和关节状态—预测动作—执行—重新观察”的闭环评测。RLinf 官方文档也把 RoboTwin 评测作为独立入口[18]。在本文系统中，OpenVLA-OFT 生成动作，RoboTwin/SAPIEN 运行物理环境，RLinf 组织评测进程和计算设备。

表 3 上游软件与本文自定义部分的边界

层次	上游提供	本文增加或修改
Panthera 资产	官方 ROS 2 仓库提供 URDF、网格、关节限位和夹爪联动关系[19]。	转换网格路径，生成 RoboTwin 配置和 SRDF；使用官方非奇异姿态作为仿真初始姿态，并记录来源提交与文件校验值。
RoboTwin 任务	任务生命周期、相机、数据记录和 SAPIEN 接口。	增加竖直黄色圆柱、黄色槽座、单 Panthera 程序化专家、七维动作接口、物理成功条件和手机视角场景。
数据接口	RoboTwin 原始轨迹记录与 RLDS/OpenVLA 数据读取基础。	加入统一仿真步时钟；每五个物理步采样一次；分开保存目标动作和实测关节状态；转换为 112 条训练和 16 条验证轨迹。
OpenVLA-OFT	预训练 VLA、LoRA 微调、连续 L1 动作头、本体状态输入和动作分块。	增加 Panthera 的七维定义、25 帧动作块、独立归一化统计，并修复本次训练遇到的学习率衰减覆盖问题。
RLinf 评测	模型 worker、环境 worker、多 GPU 调度和 RoboTwin 评测入口。	正确加载单独保存的连续动作头；支持七维动作；修正 GPU 编号；支持“预测 25 帧、只执行前 18 帧”；绑定三处 episode 上限并保存视频和逐帧记录。

接入过程中发现四个会使评测失真的软件问题：评测程序没有加载连续动作头；GPU 编号经过两次映射后选错显卡；模型端和仿真端对每次执行多少帧的理解不同；不同配置项对一条轨迹的最长步数规定不一致。修正后，项目先用离线样本核对动作值，再比较命令目标与仿真关节的实测值。受上述问题影响的运行被标记为无效，不用于判断模型能力。

5 Step by step: 初学者怎样走完一次仿真实验

本节按数据产生的顺序解释一条最小实验链。读者应先在仿真中理解每一步，再考虑真实机械臂。下面的命令面向本项目已经配置好的 Lab 工作站，属于项目内操作教程。完整的通用安装和公开复现包仍待整理。路径中的 `/data/lyy/panthera-v1a` 是实验工作目录。步骤 4 和步骤 6 会长时间占用 GPU；初次阅读时可以先查看已经保存的数据和日志，无需重新运行。实际运行前还应确认不会占用他人的 GPU。

步骤 1：先认清输入、输出和一个采样帧

假设某一时刻的状态为 $[0.10, 1.55, 1.62, -0.03, 0.02, 0.08, 0.90]$ 。前六个数是六个关节角，单位为弧度；最后一个数是归一化夹爪开度。值 0 和 1 对应哪一端由本项目的数据定义决定，不能根据数字大小猜测“开”或“合”。与这个状态一起保存的还有一张图像、任务文字和专家给出的下一步七维目标。模型训练的基本样本就是这种“观察—目标动作”配对。

检查点：在继续前，读者应能回答：本文的七个数表示六个关节目标和一个夹爪目标。末端位置与四元数属于另一种动作表示。相同的“七维”在别的机器人数据集中可能有完全不同的含义。

步骤 2：构建并进入软件环境，但先不训练

```
cd /data/lyy/panthera-v1a
bash bootstrap_lab_v1a.sh      # 首次构建；重复运行会检查已完成阶段
source activate_lab_v1a.sh
nvidia-smi
python -c "import torch, sapien, mplib; print(torch.cuda.device_count())"
```

构建脚本以普通用户创建 Miniforge/Python 环境，克隆固定版本的 RLinf 和 RoboTwin，并下载仿真资产；它会记录各阶段是否完成。激活脚本加载这一环境，后两条命令只检查 GPU 和主要仿真依赖。整个过程不启动机器人。项目不使用 Docker，日常实验也不使用管理员账户。

停止条件：如果 Python 不能导入 SAPIEN 或 MPLib，或者预定 GPU 上已有他人进程，不要继续启动采集或训练。

步骤 3：先运行程序化专家，证明任务本身可完成

```
bash run_lab_panthera_phone_oracle.sh
```

该脚本把 Panthera、圆柱、槽座和相机载入 RoboTwin。程序化专家读取仿真真值，用逆运动学和 MPLib 规划抓取、搬运和插入路径，再由 SAPIEN 实际计算接触与运动。这一步验证任务几何的可达性和成功判据的可实现性。VLA 尚未参与。当前固定场景和随机化检查各为 4/4。

检查点：必须同时查看退出码、成功条件和视频。规划成功只说明找到了关节路径；视频和物理指标用于确认圆柱最终稳定插入。

步骤 4：采集专家示范，并检查每个字段

```
bash run_lab_robotwin_panthera_phone_sft_dataset.sh
```

该脚本可在多个 GPU 上生成 128 条成功轨迹。每五个 250 Hz 物理步保存一次样本，所以学习数据的频率为 50 Hz。每个样本分别保存 RGB 图像、七维实测状态、七维目标动作、仿真步数和仿真时间。失败尝试不能冒充成功示范，训练集也禁止使用把圆柱强制附着到夹爪的辅助约束。

检查点：确认图像能看到机械臂和任务区，动作与状态都是七维，时间严格递增，轨迹没有中间采样空洞，并抽查抓取、搬运、插入和释放画面。

步骤 5：转换为 RLDS，并做一次最小训练检查

```
bash run_lab_panthera_phone_sft_pipeline.sh
```

转换器把 128 条轨迹分成 112 条训练轨迹和 16 条验证轨迹，计算每个动作维度的数据范围，并把动作缩放到便于训练的数值范围，这一步称为动作归一化。随后，每个观察与后续 25 个动作组成 25×7 训练目标。脚本还运行一步优化器检查：真正取出一批图像、状态和动作，完成预测、误差计算和一次参数更新。这个检查只能证明数据与模型接口连通，不能证明训练后的策略会成功。

步骤 6：用 OpenVLA-OFT 做监督微调

```
PANTHERA_SFT_GPUS=1,2,3 \  
bash run_lab_openvla_phone_sft_25x7_10k.sh
```

脚本通过 `torchrun` 在三张 GPU 上启动 OpenVLA-OFT。RLinf 要到闭环评测阶段才会启动。模型读取图像、任务文字和七维状态，连续动作头预测未来 25 个七维动作。训练日志中的 L1 误差是预测动作与专家动作逐项绝对差的平均值；任务成功率需要在下一步的闭环仿真中单独测量。

停止条件：出现非有限损失、训练进程异常退出、模型组件缺失或学习率与计划不符时，保留日志并停止后续评测。

步骤 7：用 RLinf 做闭环评测

```
PANTHERA_POLICY_MODEL=/data/lyy/panthera-vla/runs/待评测模型 \  
PANTHERA_EVAL_GPUS=1,2 \  
PANTHERA_ACTION_CHUNK=25 \  
PANTHERA_EVAL_EXECUTION_HORIZON=18 \  
bash run_lab_panthera_phone_policy_eval.sh
```

RLinf 在两个 GPU 上分别运行模型 worker 和 RoboTwin 环境。每轮读取新观察，模型预测 25 帧，环境只执行前 18 帧，然后重新观察。正式运行器会先生成一份最终解析配置，核对模型路径、七维接口、动作块、GPU、seed 和三处 episode 步数上限，再启动评测。

检查点：不要只看“成功率”。还要确认连续动作头确实被加载、命令值传到了仿真关节、每个 seed 都产生视频和轨迹、评测结束后没有残留 Ray 或 GPU 进程。

步骤 8：按失败发生的位置读结果

先检查机器人是否离开初始姿态，再依次检查接近、闭爪、抬升、搬运、对准、插入、松爪和撤离。若机器人根本不动，应先查模型文件和动作接口；若已经搬运但圆柱倾斜，应查夹持位置和开环执行长度；若圆柱进入槽口后失败，应查接触、插入深度和释放时速度。只有开发集达到预设门槛，且最终留出集也通过，才进入真实图像影子模式。影子模式把预测动作写入日志，机械臂不会收到这些动作。

初学者应得到的结论：一项完整的具身智能实验包含机器人建模、任务验证、数据检查、模型训练、闭环执行和结果评测。上述证据必须依次成立。任何一层出错，都可能产生一个看似正常但没有意义的成功率。

6 场景、数据与训练

6.1 仿真画面与真实画面的初步匹配

真实工作台曾由手机广角相机通过 SRT (Secure Reliable Transport) 协议发送网络视频。第一版仿真画面只拍到机械臂末端，而真实画面中机械臂位于左侧、任务物体位于中央。由于两种画面的主体位置差异明显，第一版的 128 条示范被归档为诊断数据，没有用于正式训练。

第三版场景（下文简称 v3）把仿真相机的垂直视场角设为 75° ，先渲染宽画面，再取中央 4:3 区域并缩放为 320×240 像素。调整后，机械臂位于画面左侧，圆柱和槽座位于中央。训练场景还在限定范围内改变光照、背景、物体位置和相机姿态。这种做法称为域随机化，其目的是让模型不要只记住一种固定画面[9]。

这里的“匹配”只表示画面构图大致相似。手机已经从实验台拆下，项目尚未测量最终相机的内参、相机相对机器人基座的位置，以及圆柱和槽座的精确尺寸。当前结果应称为构图近似。精确的仿真—真实对齐仍需重新安装相机并完成几何测量；背景随机化只能增加外观变化。

6.2 专家数据

程序化专家能够读取仿真中的精确圆柱和槽座位置。它先用运动规划与逆运动学计算机械臂路径，再让物理仿真器实际执行抓取、搬运、插入和松爪。只有执行结果满足成功条件的轨迹才作为专家示范进入数据集；失败轨迹只用于查找任务设计问题。

v3 数据集包含 128 条成功轨迹和约 105,662 个采样帧，其中 112 条用于更新模型参数，另外 16 条用于训练期间计算验证误差。本文尚未统计程序化专家为得到这 128 条成功轨迹总共尝试了多少次，因此不能用该数据集估计随机场景本身的难度。数据随后按照 RLDS 格式保存，本文使用的数据集版本号为 3.0.0。RLDS 是保存多条轨迹及其帧顺序的一种数据组织方式[10]，它不改变图像、关节角或动作本身。

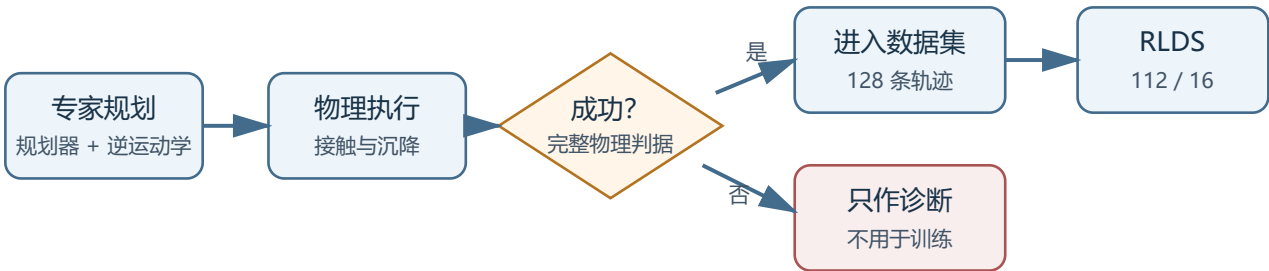


图 2 专家数据的生成过程。只有通过物理仿真成功条件的轨迹进入训练数据；失败轨迹另行保存。

6.3 OpenVLA-OFT 与动作块

OpenVLA 先从大规模图像、文字和机器人示范中学习通用表示[2]。本项目使用 OpenVLA-OFT 的连续动作版本进行监督微调[11]：输入为当前图像、固定任务文字和七维机器人状态，输出层直接预测关节角与夹爪开度。这个专门输出连续动作数值的网络部分称为“连续动作头”。

当前模型一次预测未来 25 帧，每帧包含 7 个动作数值。忽略训练程序中的掩码和归一化细节后，可以用下式直观表示一个动作块的平均绝对误差：

$$L_{L1} = (1 / 175) \sum_{k=0 \dots 24} \sum_{j=1 \dots 7} |\hat{a}_{t+k,j} - a_{t+k,j}|$$

式中 $\hat{a}$ 是模型预测， a 是专家动作， $25 \times 7 = 175$ 。这个式子用于解释损失含义，不声称完整复现训练代码中的每个实现细节。离线误差是在专家已经到达的状态上计算的。连续执行时，模型的一次错误会改变下一张图像，后续输入可能不再像训练数据。这就是模仿学习中的状态分布偏移问题[12]。

6.4 训练设置

表 4 汇总影响结果解释的主要设置。每张 GPU 的批量大小为 1，三张 GPU 同时训练，因此每个优化步骤合计处理 3 个样本。模型采用 LoRA 微调，低秩矩阵的秩为 32。10k 模型使用 5×10^{-4} 的学习率；从 10k 续训到 15k 时降为 5×10^{-5} ，随后计划从 15k 以 1×10^{-5} 续训到 20k。训练程序每 1,000 步计算一次验证误差。

“使用 LoRA” 需要进一步说明哪些参数会更新。OpenVLA 原有视觉编码器、视觉—语言连接层和语言模型中的基础权重保持冻结；训练程序在这些网络的线性层旁加入可训练的 LoRA 低秩参数。除此之外，本项目新建的连续 L1 动作头和七维本体状态投影器也参与训练。由于 `use_film=false`、`use_diffusion=false`，本实验没有 FiLM 参数、扩散动作头或噪声动作投影器。训练结束时可以把 LoRA 参数合并进基础模型权重；训练期间的 70 亿基础参数仍保持冻结。上述模块选择可由 OpenVLA-OFT 的公开训练代码核对[17]。

表 4 训练与数据设置

项目	本文实验设置
输入	场景图像保存为 320×240，送入模型前缩放为 224×224；另有固定语言指令和 7 维机器人状态
输出	25×7 的绝对关节与夹爪动作块
数据划分	112 条训练轨迹、16 条验证轨迹；另有独立的闭环开发场景和最终测试场景
优化批量	GPU 1、2、3 各 1 个样本；有效批量为 3；梯度累积为 1
微调方法	LoRA 的矩阵秩为 32；图像增强开启；使用连续 L1 动作头
时间频率	动作数据与执行频率为 50 Hz；每次预测 0.5 s，实际执行前 0.36 s

6.5 闭环评测集合与成功条件

训练结束后，模型在仿真中连续控制机器人。项目使用三组场景。第一组只有 4 个场景，特意包含已知成功和失败案例，用于快速检查代码修改。它经过人为挑选，只能用于诊断。第二组含 16 个开发场景，随机种子为 200001 ~ 200016，可以反复运行并据此修改模型或规则，本文简称 dev16。第三组是种子 300001 ~ 300016 的 16 个最终测试场景，本文简称 final16；只有方案停止修改后才会首次运行。

某一采样帧同时满足下列条件时，评测程序把该帧记为通过：圆柱在水平两个方向上的位置误差均不超过 10 mm，高度误差不超过 12 mm，圆柱轴线偏差不超过 5°，插入深度至少为 30 mm，线速度不超过 0.025 m/s，角速度不超过 0.35 rad/s，夹爪已经打开，并且夹爪与圆柱之间的接触冲量已经归零。

这些数值是当前仿真实验采用的工程阈值。由于真实圆柱、孔和装配间隙尚未完成精确测量，本文不能证明这些阈值等同于真实任务的合格公差。重新搭建实验台后，需要用实测尺寸重新审查它们。

评测程序还区分两个轨迹级指标：“曾经通过”表示轨迹中至少有一帧通过上述条件；“结束时通过”表示最后一帧仍然通过。本文现有的 4/16 和 11/16 使用前一种指标。相同实验的“结束时通过”数量尚未汇总，因此这两个数字只描述轨迹中出现过合格状态。

为表示 16 条轨迹带来的抽样不确定性，图 4 同时给出二项比例的 Wilson 95% 置信区间。它表示：如果把这些场景近似看作从某个固定分布中独立抽取，有限样本允许的成功率范围仍然很宽。实际 dev16 是固定且被反复查看的开发集，所以该区间只能帮助读者理解样本量，不能把开发结果变成总体性能保证。

7 滚动时域与混合控制

第一版模型只预测未来 5 帧，即 0.1 s。专家示范开始时先调整夹爪，机械臂关节在几十帧内几乎不动。因此，当模型只看未来 5 帧时，“继续不动”在训练数据中可能是正确答案，模型也容易反复停在这一状态。把预测长度增加到 25 帧，即 0.5 s 后，动作块中开始包含机械臂离开初始位置的动作。ACT 等精细操作研究也采用一次预测多个连续动作的方法[13]。

模型每次生成 25 帧预测。执行程序采用滚动时域：先执行前 18 帧，即 0.36 s，丢弃剩余 7 帧，然后读取新的图像和关节状态并再次预测。较短的实际执行段让机器人更早看到动作造成的新结果，减少抓取和碰撞误差长时间累积的机会。

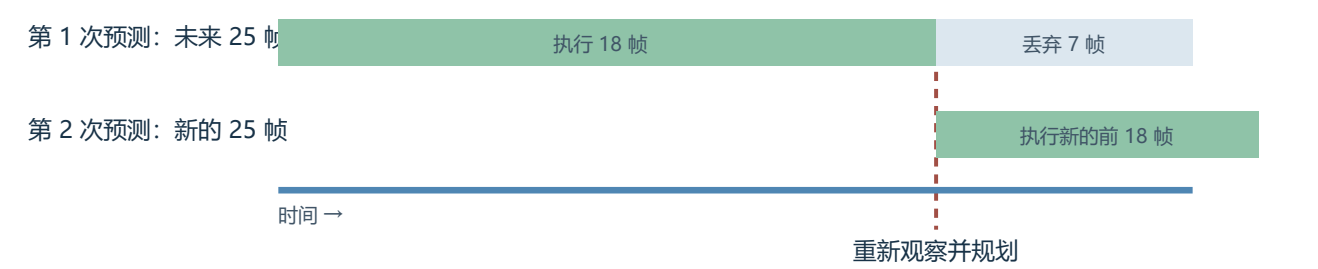


图 3 模型预测 25 帧，但只执行前 18 帧。第 18 帧后重新观察，因此旧预测的最后 7 帧不会继续执行。

25 帧模型能够在部分场景中抓住并搬运圆柱，但搬运时间较长时，圆柱会在夹爪中逐渐倾斜。为判断最后的接触阶段是否可以由传统控制方法改善，项目增加了一个确定性末端技能。当夹爪开度表明圆柱可能已被夹住、机械臂末端进入预先标定的槽位附近，并且 VLA 输出松爪请求时，程序停止采用 VLA 后续动作，改为依次执行固定的定姿、对齐、分段下插、等待物体稳定、松爪和撤离。

夹爪开度在这里充当抓取代理：圆柱挡住夹爪继续闭合时，开度会停在一个特定范围。这个信号只说明夹爪中可能夹有物体，无法判断圆柱姿态。末端技能读取夹爪开度、机械臂末端位姿和事先标定的槽位；仿真器提供的圆柱精确位置只写入实验日志，供事后分析。后文的 11/16 因此评价 VLA 与末端技能组成的完整系统。

8 实验结果

结果分为任务检查、两场景诊断和开发集评测。表中“4 个场景”与“2 个场景”只用于快速定位问题，样本太少，不能解释为模型的一般成功率。dev16 可以比较同一批场景上的方案变化，但由于开发过程反复查看了这些结果，它也不能替代尚未运行的最终测试集。

表 5 截至 2026 年 9 月 15 日已经完成的实验

实验阶段	评测	结果	结论
程序化专家，固定场景	4 个场景	4/4	证明任务程序和规划路径在这 4 个场景中可以执行。
程序化专家，随机场景	4 个场景	4/4	证明随机化后的 4 个检查场景也可以执行；该阶段尚未运行 VLA。
v3 示范数据	128 条成功轨迹	格式检查通过	112 条训练、16 条验证；未统计生成这些成功轨迹的总尝试次数。
5×7 动作块，5k 与 20k 检查点	每次 2 条诊断轨迹：1 条来自训练场景，1 条来自新场景	两种检查点均为 0/2	模型反复输出示范开头的近静止动作；单纯增加训练步数没有解决。
25×7 动作块，10k 检查点	同样的 2 条诊断轨迹	0/2	模型已经能够接触、夹持和搬运，但在圆柱倾斜或释放时失败。
25×7，15k 检查点，只使用 VLA	16 个开发场景	曾经通过 4/16	同一批开发场景上的组合系统对照。
25×7，15k 检查点，加确定性末端技能	同一组 16 个开发场景	曾经通过 11/16 (68.75%)	组合系统的开发结果；95% 置信区间为 44.4% ~ 85.8%，结束时通过数量尚未报告。
从 10k 以较高学习率续训到 30k	16 个开发场景	曾经通过 1/16	性能明显下降，因此不再采用该检查点。
把末端技能的固定横向偏移从 4.5 mm 改为 7 mm	4 个定向检查场景	曾经通过 1/4	没有改善结果，恢复为 4.5 mm。
最终测试集	16 个从未用于调参的场景	尚未运行	开发结果没有达到预先规定的 12/16 启动条件。

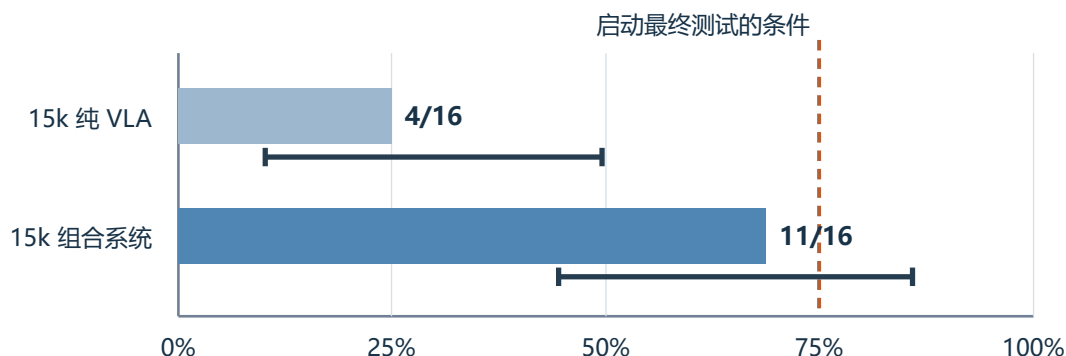


图 4 同一组 16 个开发场景上的“曾经通过”比例。短横线为 Wilson 95% 置信区间；橙色虚线表示启动最终测试的工程条件。

纯 VLA 的 4/16 对应 25.0%，其 Wilson 95% 置信区间为 10.2% ~ 49.5%；组合系统的 11/16 对应 68.75%，区间为 44.4% ~ 85.8%。两个区间都很宽，说明 16 条轨迹只够作阶段性判断。由于两种方法运行在同一批场景上，后续还应在两个总数之外公开每个 seed 的配对结果。

9 失败分析与研究认识

第一次 0/16 来自评测程序错误。OpenVLA-OFT 把连续动作头单独保存。最初的 RLinf 评测程序遗漏了这个文件，实际运行了原模型的离散动作输出。该次运行所用控制器与训练目标不一致，所以结果被标为无效。程序修改后，项目用五个离线样本逐项比较预测数值，并检查仿真关节是否到达所发送的目标。

离线误差下降没有自动变成任务成功。5 帧模型续训到 20k 后，验证集上的平均动作误差下降，但仿真机械臂仍在初始位置附近小幅运动。原因之一是训练误差只在专家走过的状态上计算；仿真闭环中，一次空抓就会产生训练集中少见的图像和夹爪状态，后续误差会继续累积。

25 帧模型把主要失败位置从开场推进到了接触阶段。它不再停在初始位置，能够完成接触、夹持和一段搬运。剩余失败常见于四种情况：夹爪没有抓住圆柱中心，圆柱在搬运中逐渐倾斜，圆柱边缘撞到槽壁，或者松爪时圆柱仍有较大速度。把末端技能的统一横向偏移从 4.5 mm 改为 7 mm 后结果更差，说明不同场景的误差方向并不相同，不能用一个固定平移修正所有失败。

组合系统用于定位末端失败来源。VLA 负责从图像决定抓取和搬运，固定程序负责几何已知的最后阶段。组合结果高于纯 VLA，说明末端控制方法会显著影响任务结果。11/16 是这一具体组合的成绩；其中的插入动作由固定程序完成。

10 逐级评测和安全边界

项目采用逐级评测。每次修改先用 4 个已知场景检查程序是否退化，通过后再运行 16 个开发场景。开发集中至少 12 条轨迹曾经满足成功条件时，才允许首次运行 16 个最终测试场景。75% 是项目事先规定的工程停止条件；统计显著性和最终测试仍需分别报告。



图 5 逐级评测顺序。即使仿真最终测试通过，也要先在不控制机械臂的条件下检查真实图像输入。

4 个检查场景使用种子 200001、200006、200009 和 200014，其中两个原本成功，两个原本失败，所以它们是人为选择的调试样本。完整开发集使用 200001 ~ 200016，最终测试集使用此前没有参与调参的 300001 ~ 300016。每次仿真结束后还要保存视频和逐帧数值记录，并确认并行仿真进程已经退出。进入真实环境后的第一步称为影子模式：程序读取真实相机和机器人状态并计算动作，但不把动作发送给机器人。

11 局限性与下一步

- 11/16 是反复参与调试的开发集结果，而且目前只报告“曾经通过”；结束时和连续保持成功的数量尚未给出。
- 16 条开发轨迹的统计不确定性较大。最终测试集没有运行，因此尚无独立的泛化结果。
- 训练数据只有 128 条成功示范，任务文字、物体种类和场景变化有限；生成成功示范之前的总尝试次数也未统计。
- 手机相机与控制盒已拆下。仿真只完成构图近似，尚未测量相机内外参、真实圆柱尺寸、槽尺寸和装配间隙。
- 仿真中的摩擦、柔顺性和夹爪接触不能完全代表真实设备；系统也没有力或触觉传感器直接测量槽壁接触。
- 确定性末端技能依赖固定槽位标定，不能处理任意位置和形状的插入任务。
- 当前论文材料只支持项目内部追踪。正式发布前需要冻结仓库提交，列出 RLinf、RoboTwin、模型和补丁版本，并保存环境、随机种子、配置与输出文件的校验值。

已经完成的三个末端规则修改——7 mm 固定偏移、分两次松爪和在较浅位置松爪——都没有增加成功场景，因此已停止继续枚举末端几何常数。当前计划从稳定的 15k 检查点以更低学习率续训到 20k，再按相同标准运行开发集。只有方案冻结后，开发集和最终测试集都达到预设条件，才会重新安装手机、测量相机和物体几何，并先运行不发送动作的影子模式。更长期的研究可以补充失败状态下的纠正示范，使用 DAgger 收集模型实际会遇到的状态[12]，增加腕部相机或力传感器，并与 ACT、Diffusion Policy 和传统视觉伺服方法比较。

12 结论

本项目已经完成单 Panthera 仿真任务、专家数据采集、OpenVLA-OFT 微调和连续控制评测。实验清楚地区分了三类问题：短动作块使模型停在示范开头；较长动作块改善了抓取和搬运；圆柱与夹爪、槽壁之间的接触仍造成插入失败。评测程序本身也曾产生无效结果，说明在评价学习模型之前必须先验证模型文件、动作数值和仿真执行是否一致。

目前最好的已完成结果是：15k VLA 与确定性末端技能组成的系统，在 16 个开发场景中有 11 条轨迹曾至少一次满足成功条件。启动最终测试需要 12/16，结束时稳定成功率也尚未汇总。本文的有效结论限定为“组合系统在开发集上比纯 VLA 表现更好”。VLA 独立插入能力、最终仿真测试以及真实机械臂迁移仍处于待验证状态。

13 数据、代码与复现状态

本地仓库保存了仿真环境覆盖代码、训练与评测启动脚本、评测指标实现和阶段说明。构建脚本当前固定 RLinif 提交 a3816b596478...、RoboTwin 提交 0008ae6800df...，Panthera 资产来自官方 ROS 2 仓库提交 b08633d6c5bc...。Lab 工作站的 /data/lyy/panthera-vla/ 保存模型、逐帧轨迹、日志和视频。

这些材料可以支持项目内部追踪，目前尚未形成不可变的公开复现包。正式发布时还需要固定本项目与 OpenVLA-OFT 的完整提交号，记录 Python、CUDA 和驱动版本，公开训练配置、数据划分、全部评测 seed、补丁清单、模型文件校验值，以及 dev16 的逐 seed 配对结果。

项目内部记录： README.md; docs/01_next_stage_vision_teleop_todo.md; docs/05_software_acceptance.md; docs/06_data_collection_cases.md; docs/08_lab_experiment_progress_2026-09-13.md; docs/09_sim_real_scene_alignment.md; docs/10_openvla_closed_loop_failure_analysis.md。模型、轨迹、视频与报告保存在 Lab 的 /data/lyy/panthera-vla/。这些路径用于项目内部核查，公开发布时仍需转换成稳定、可下载的论文附件。

参考文献

- [1] Brohan, A., et al. (2023). [RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control](#).
- [2] Kim, M. J., et al. (2025). [OpenVLA: An Open-Source Vision-Language-Action Model](#). Proceedings of the 8th Conference on Robot Learning, 2679–2713.
- [3] Lynch, K. M., & Park, F. C. (2017). [Modern Robotics: Mechanics, Planning, and Control](#). Cambridge University Press.
- [4] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). [Robot Operating System 2: Design, architecture, and uses in the wild](#). Science Robotics, 7(66), eabm6074.
- [5] Lugaresi, C., et al. (2019). [MediaPipe: A Framework for Building Perception Pipelines](#). arXiv:1906.08172.
- [6] Chen, T., et al. (2025). [RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation](#). arXiv:2506.18088.
- [7] Xiang, F., et al. (2020). [SAPIEN: A SimULATED Part-Based Interactive ENvironment](#). CVPR 2020.
- [8] Yu, C., et al. (2025). [RLinf: Flexible and Efficient Large-scale Reinforcement Learning via Macro-to-Micro Flow Transformation](#). arXiv preprint, arXiv:2509.15965.
- [9] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). [Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World](#).
- [10] Ramos, S., et al. (2021). [RLDS: an Ecosystem to Generate, Share and Use Datasets in Reinforcement Learning](#).
- [11] Kim, M. J., Finn, C., & Liang, P. (2025). [Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success](#). Robotics: Science and Systems XXI. doi:10.15607/RSS.2025.XXI.017.
- [12] Ross, S., Gordon, G. J., & Bagnell, J. A. (2011). [A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning](#). AISTATS 2011.
- [13] Zhao, T. Z., Kumar, V., Levine, S., & Finn, C. (2023). [Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware](#).
- [14] Open X-Embodiment Collaboration. (2023). [Open X-Embodiment: Robotic Learning Datasets and RT-X Models](#).
- [15] Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., & Song, S. (2023). [Diffusion Policy: Visuomotor Policy Learning via Action Diffusion](#). Robotics: Science and Systems XIX.
- [16] Hutchinson, S., Hager, G. D., & Corke, P. I. (1996). [A Tutorial on Visual Servo Control](#). IEEE Transactions on Robotics and Automation, 12(5), 651–670.
- [17] Kim, M. J., Finn, C., & Liang, P. [OpenVLA-OFT: vla-scripts/finetune.py](#). Official implementation, accessed 2026-09-15.
- [18] RLinf Contributors. [RoboTwin 评测指南](#). RLinf official documentation, accessed 2026-09-15.
- [19] HighTorque Robotics. [Panthera-HT ROS 2](#). Official robot description and ROS 2 driver repository, accessed 2026-09-15.